SACHIN KUMAR(ME23B065), ROHIT B(ME23B064), DHISANTH(MM23B043)

CHESS BEHAVIOR PREDICTION: INTEGRATING PLAYER PROFILES AND TEMPORAL GAME CONTEXT INTO POSITIONAL EVALUATION

ABSTRACT

Traditional chess engines evaluate positions primarily through static heuristics or deep neural search trees that look for objectively optimal moves based solely on the current board state. However, such evaluation mechanisms entirely overlook human cognitive factors, player skill differentials, and temporal psychological fatigue. This paper presents a novel approach to chess analysis by integrating multi-modal inputs—combining high-dimensional spatial board tensors with hand-crafted scalar features including player Elo ratings, material balance, move sequence history, and tactical mobility metrics. Instead of predicting the standard objective evaluation score, our system predicts the behavioral quality category of the subsequent human move (classified as excellent, good, inaccuracy, mistake, or blunder). By conditioning position evaluation on contextual parameters like player Elo differences and game momentum, the model establishes a personalized, cognitively-aware evaluation framework. This shifts the focus from purely objective engine lines to descriptive behavioral prediction, providing deep insights into where specific tiers of players lose cognitive focus and commit critical errors.

1. INTRODUCTION

The domain of computer chess has advanced phenomenally over the past several decades, evolving from Shannon-type alpha-beta minimax search heuristics to contemporary deep reinforcement learning architectures such as AlphaZero and Stockfish NNUE. Despite their unprecedented tactical supremacy, these systems operate under a rigid paradigm: they assume error-free execution and evaluate board states purely from an objective mathematical perspective. They calculate an absolute positional evaluation under the implicit assumption that both opponents are deterministic, infinitely capable entities.

However, human chess is intrinsically psychological and boundedly rational. A complex positional configuration that guarantees a straightforward win for a Grandmaster (GM) represents an impenetrable, high-variance hazard for a novice player. Traditional engines provide no differentiation between these scenarios, presenting a flat evaluation metric that fails to capture human cognitive vulnerability, mental fatigue, or situational pressure.

To bridge this gap, we propose an innovative behavioral prediction framework that incorporates a dual-stream input layout. By feeding both a structured tensor representation of the current board state and a comprehensive array of contextual scalar metrics into a machine learning classifier, our system models the likely quality of the player's upcoming move. Rather than isolating the board topology, our framework dynamically accounts for player skill tiers, temporal game phase indicators, and historical momentum shifts.

1.1. Background

Modern computerized chess analysis relies on converting a board state into a numerical value representing a centipawn advantage or a forced mate sequence. This translation is carried out either by handcrafted evaluation functions that weigh piece values, king safety, and pawn structures, or by deeply trained neural networks mapping raw bitboards to positional scores. When a human player reviews their game using these traditional tools, the engine identifies deviations from the top computed choices, categorizing human errors relative to an unattainable, hyper-rational standard.

While this objective benchmark is invaluable for theoretical study, it lacks explanatory power regarding human performance. Human decision-making in chess is a sequential cognitive process influenced by time pressure, tactical complexity, and psychological momentum. Integrating these behavioral dimensions requires moving beyond static positional snapshots toward a holistic, context-driven representation of the match environment.

1.2. Literature Review

The intersection of human behavior and computer chess has gained traction through recent attempts to humanize AI play styles. Notable projects like *Maia Chess* have utilized the Leela Chess Zero architecture trained on human games of specific Elo ratings to predict the exact move a human of a given tier would play. While Maia demonstrates that human moves are highly predictable when conditioned on rating-specific datasets, its internal architecture still functions primarily as a policy network mapping board states to a single move probability distribution.

Complementary behavioral research indicates that human error rates are non-uniformly distributed across a game's timeline. Cognitive fatigue degrades decision-making quality as the move count advances, and sharp shifts in material balance or tactical mobility frequently induce psychological triggers that lead to cascading blunders. Existing models, however, rarely ingest these extrinsic meta-features directly alongside the spatial board matrix, leaving an open avenue for multi-modal networks that synthesize board topography with historical player metadata.

1.3. Research Gap and Challenges

A significant research gap exists in standard position evaluation frameworks: they completely isolate the board state from the agents executing the moves. No existing mainstream analytical framework dynamically alters its behavioral expectation or error-prediction threshold based on the specific matchup context, such as a major rating discrepancy between the players.

The primary challenge in resolving this gap lies in the effective fusion of heterogeneous data sources. A chess position is naturally represented as a high-dimensional spatial tensor ($8\times8\times C$), whereas historical game momentum and player demographics are unstructured or scalar time-series variables. Devising an architecture that effectively balances spatial board features against contextual scalar dynamics without allowing one stream to saturate or obscure the other remains a non-trivial machine learning challenge. Furthermore, extracting a concise numerical proxy for game history and emotional momentum introduces complex feature-engineering trade-offs.

1.4. Problem Statement

Given an arbitrary chess board configuration represented as a multi-channel spatial tensor $\mathcal{T}$, alongside an asymmetric set of contextual scalar features $\mathbf{x}_{\text{scalar}}$ encompassing player profiles, chronological game indicators, and current tactical strain, the objective is to learn a mapping function $f : (\mathcal{T}, \mathbf{x}_{\text{scalar}}) \rightarrow \mathbf{y}$. Here, $\mathbf{y}$ is a probability distribution over five ordinal move-quality categories:

$$\mathcal{Y} = \{\text{Excellent, Good, Inaccuracy, Mistake, Blunder}\} \quad (1)$$

The model must accurately anticipate the cognitive execution threshold of the active player, predicting whether the situational context and board complexity will culminate in a structural blunder or an optimal continuation.

1.5. Objectives

The primary objectives of this research project are outlined as follows:

1. To engineer a robust multi-modal data pipeline that transforms standard Portable Game Notation (PGN) data into coupled spatial board tensors and multi-dimensional contextual scalar vectors.

2. To implement an expressive scalar feature suite that characterizes player skill differentials (ΔElo), temporal fatigue (move_number, halfmove_clock), tactical complexity (mobility, is_check), and momentum shifts (prev_eval, prev_delta).

3. To design and train a classifier capable of synthesizing these distinct streams to predict human move quality categories, establishing a personalized analysis framework.

4. To quantify the specific situational thresholds and game contexts where players across different Elo tiers demonstrate a pronounced loss of cognitive focus.

2. METHODOLOGY

2.1. Data Generation and Feature Description

To capture both spatial board configurations and the cognitive behavioral context of human players, a custom multi-modal data mining pipeline was engineered. The process streams raw historical chess games from the March 2017 Lichess public database, which is stored as a compressed Zstandard (`.zst`) Portable Game Notation (PGN) file. A total of 5,000 matches were iteratively decoded using the `python-chess` library.

For every individual sequential move within a match, the pipeline dynamically extracts the current board position using its Forsyth-Edwards Notation (FEN) token. To acquire real-time position evaluations, the FEN string is transferred via standard input/output streams to an external Stockfish execution thread operating at a fixed analysis depth of 10. The engine's centipawn (`cp`) evaluation is extracted and normalized to match the active player's perspectives.

The spatial board topologies are isolated and exported into a native three-dimensional compressed NumPy matrix array (`.npy`), whereas the complementary behavioral meta-features are synchronized into a tabular Apache Parquet serialization format for high-speed machine learning integration.

The extraction framework separates information into spatial structural layers and a series of hand-crafted contextual scalar metrics. The extracted features and their expected outputs across different scenarios are categorized as follows:

- **Spatial Positional Tensor (`tensor_12`):** An $12 \times 8 \times 8$ binary tensor tracking piece positions. The 12 channels correspond uniquely to the binary maps of each piece type (Pawns, Knights, Bishops, Rooks, Queens, and Kings) separated by color.

- **Temporal & Game State Progress:** Includes `move_number` (chronological turn index starting at 1) and `halfmove_clock` (resets to 0 after any pawn move or piece capture; increments otherwise to track the 50-move draw rule limit). The `turn` column outputs 1 for White and 0 for Black.

- **Tactical Context & Structural Boundaries:** Tracks `is_check` (outputs 1 if the current player's king is under direct attack, forcing defensive actions; 0 otherwise) and `mobility` (integer count of total active legal moves available, dropping significantly in cramped or highly restricted setups). The `material_balance` calculates net piece values, outputting 0 for equal material, positive values when White is up material, and negative values when Black leads.

- **Asymmetrical Player Demographics:** Features `elo_diff` (White's rating minus Black's rating, outputting a positive value when White is the higher-rated favorite and negative when Black is favored) and `avg_elo` (a float value defining the baseline skill bracket of the match, separating novice, intermediate, and master-level matches).
- **Game Progress Context:** Captures `en_passant` (outputs -1 if no en passant capture is legally available; otherwise maps the destination square coordinate to a unique integer space). Boolean flags for remaining castling allowances (`castle_K`, `castle_Q`, `castle_k`, `castle_q`) output 1 if the specific kingside or queenside castling right is active for that color, and 0 if it has been permanently lost due to a king or rook movement.
- **Historical Momentum Metrics:** Employs `prev_eval` (the static centipawn evaluation before the current turn, where positive values favor White and negative values favor Black) and `prev_delta` (the evaluation shift caused by the opponent's immediate prior choice, reflecting whether the player is reacting to a recent blunder or defending against an optimal move sequence).

To enforce strict causal ordering during model training, a clear boundary is established regarding when data features are calculated. A distinct set of metrics captured within our pipeline are structural consequences generated *after* a move has been finalized on the board.

> **Important Constraint Note:** The features `eval`, `delta`, `piece_moved`, `is_capture`, and `gives_check` cannot be utilized as training inputs for our predictive features. Because these features represent target-leaking data elements evaluated downstream from the user's execution choice, utilizing them as model features would invalidate the behavior-prediction framework.
>
> Instead, the `delta` column (Δ = eval$-$prev_eval) serves strictly as a labeling anchor, passed into a classification block to map data boundaries into categorical move classifications (*excellent*, *good*, *inaccuracy*, *mistake*, or *blunder*).

Note: Please refer to `Group14_Source_Code1.ipynb` for the complete data generation scripts, machine learning pipeline, and corresponding results.

2.2. Baseline Modeling via Extreme Gradient Boosting (XGBoost- only on scalar features)

To evaluate the predictive boundaries of the hand-crafted scalar features before compounding the spatial matrices, an Extreme Gradient Boosting (XGBoost) architecture was introduced. The target output vector $\mathbf{y}$ presents a highly skewed class distribution containing a severe representation imbalance. Specifically, the baseline distribution contains $221,151$ *good* occurrences, $57,825$ *excellent* occurrences, $28,194$ *inaccuracy* instances, $22,817$ *mistake* instances, and only $7,994$ structural *blunders*.

2.2.1. Algorithmic Structure and Engineering Techniques

The architectural processing pipeline executes the following technical sequence:

1. **Label Transmutation:** Targets are compiled into discrete indices via Scikit-Learn's `LabelEncoder`. They are then transformed into a binary representation matrix through categorical assignment utilities.
2. **Data Separation and Stratification:** The dataset is partitioned into an $80/20$ train-to-test calibration ratio. To preserve the skew across partitions, stratification is enforced relative to the ground-truth index distribution: $\arg\max(\mathbf{y}, \text{axis} = 1)$.
3. **Feature Normalization:** A `StandardScaler` transforms the 15 input features to achieve zero mean and unit variance. This prevents variance discrepancies across asymmetrical scales from creating optimization friction.
4. **Model Instantiation:** An `XGBClassifier` is initialized with a multi-class softmax loss objective function (`multi:softmax`). Regularization terms are defined to mitigate over-parameterization ($\lambda = 1.0, \alpha = 0.5$). Deep decision trees (`max_depth=5`) are trained via a histogram-based algorithm (`tree_method='hist'`) across 200 gradient-boosting iterations.

2.2.2. Baseline Performance and Error Source Analysis

The initial baseline execution achieved a raw classification accuracy of 67.00%. Despite this reasonable overall accuracy, a detailed review of the classification report reveals critical performance limitations caused by class imbalance:

- **Imbalance Collapse:** The model demonstrates high predictive strength for the majority class (*good*: precision = 0.68, recall = 0.94, F1 = 0.79), but fails completely on minority classes. For instance, it yields an F1-score of 0.00 for *inaccuracy* and a recall of just 0.03 for *blunder*.
- **Primary Causes of Low Performance:** Because the optimization function prioritizes maximizing global accuracy over skewed boundaries, the network skews heavily toward the dominant class. Furthermore, isolating scalar spatial indices forces the engine to differentiate highly complex configurations using text summaries alone. Without spatial context, identical scalar profiles can lead to drastically different move qualities, causing high cross-class ambiguity.

Note: Please refer to `Group14_Source_Code2.ipynb` for the explicit script setups, loss tracking curves, and confusion matrix plots for the baseline imbalanced model.

2.3. Data Alignment via Bootstrap Oversampling (for xgboost on scalar features)

To prevent the gradient optimization steps from collapsing into majority-class predictions, the data space was altered using a bootstrap oversampling technique.

2.3.1. Resampling Implementation The strategy applies random replacement methods (`sklearn.utils.resample`) across every independent minority class slice to match the count of the majority target. This approach expands the total training footprint to a uniform distribution of 221, 151 samples per class across all categories. After balancing the target classes, the data is split, normalized, and optimized using the same core structural pipeline as the baseline model. To handle the larger data scale, hyperparameter tuning limits were increased to 500 trees (`n_estimators=500`) and tree depth limits were raised (`max_depth=10`).

2.3.2. Performance Analysis and Improvements Following the bootstrap balancing step, global classification accuracy improved to 71.00%. The balanced model exhibits several key behavioral shifts:

- **Minority Class Recovery:** The model overcomes minority class omission, yielding balanced F1-scores across previously neglected tiers (*blunder* F1 = 0.85, *inaccuracy* F1 = 0.65, *mistake* F1 = 0.69).
- **Underlying Causes of Better Accuracy:** Balancing the data forces the gradient steps to penalize errors equally across all classes, preventing the model from collapsing into majority-class guesses. Additionally, expanding the model's capacity—by raising tree counts and increasing depth limits—allows XGBoost to build more complex decision boundaries around the synthesized feature distributions.

Note: Please refer to `Group14_Source_Code3.ipynb` for the complete oversampled processing execution scripts and the resulting feature importance plots.

2.4. Multi-Modal Deep Neural Network (DNN) with Class-Weight Balancing

To leverage both the structural layout of the board and the contextual meta-features simultaneously, a multi-modal Deep Neural Network (DNN) framework was designed. Unlike the preceding models that either excluded spatial indices or relied on physical oversampling, this architecture preserves the original dataset size while handling class imbalance mathematically during loss optimization.

2.4.1. Feature Fusion and Data Balancing The integration of spatial and scalar feature spaces is executed through a deterministic concatenation pipeline:

1. **Tensor Flattening:** The multi-channel spatial chess board configurations, initially matching a four-dimensional shape of $(\text{samples}, 12, 8, 8)$, are mapped to a two-dimensional matrix space $(\text{samples}, 768)$ via a reshaping operation.
2. **Horizontal Concatenation:** The 768 flattened spatial piece features are concatenated with the 15 normalized environmental scalar attributes along the feature axis. This produces a unified representation vector $\mathbf{X}$ with a dynamic input dimensionality of 783 attributes.
3. **Mathematical Cost Weighting:** To handle the unequal target label distribution (221, 151 *good* vs. 7, 994 *blunder*), class weights are computed using a balanced bootstrap formulation:

$$w_j = \frac{N_{\text{samples}}}{N_{\text{classes}} \times N_j} \tag{2}$$

This assigns an inverse scaling factor to the loss function based on empirical class frequency, generating calculated penalty weights: *blunder* (0) = 8.46, *excellent* (1) = 1.17, *good* (2) = 0.31, *inaccuracy* (3) = 2.40, and *mistake* (4) = 2.96.

2.4.2. Architectural Topology and Hyperparameter Grid Search The deep feedforward network was constructed using a sequential layer layout. To determine the most stable training boundaries, an exhaustive hyperparameter grid search was executed over a search space of 32 independent configuration pipelines, testing structural variations across dense layer units (128, 256), dropout rates (0.3, 0.5), optimization algorithms (`Adam`, `RMSprop`), learning rates (0.001, 0.0005), and batch constraints (64, 128).

The optimal network configuration selected by the optimization search loop follows the specific structural layout detailed below:

- **Layer 1 (Input Dense):** A fully connected layer mapping the 783 integrated inputs to 256 neurons via a Rectified Linear Unit (ReLU) activation (200, 704 parameters).
- **Layer 2 (BatchNorm 1):** A batch normalization step to control internal covariate shifts (1, 024 parameters).
- **Layer 3 (Hidden Dense 1):** A fully connected layer maintaining 256 units with ReLU activation (65, 792 parameters).
- **Layer 4 (BatchNorm 2):** A second stabilization step applying mini-batch normalization scaling (1, 024 parameters).
- **Layer 5 (Dropout 1):** A regularization layer dropping 30% of active nodes to prevent over-parameterization.
- **Layer 6 (Hidden Dense 2):** A bottleneck dense tier reducing the dimensionality down to 128 units using ReLU (32, 896 parameters).
- **Layer 7 (Dropout 2):** A final structural regularization layer enforcing a 0.3 dropout rate.
- **Layer 8 (Output Layer):** A 5-unit dense classification layer computing a categorical probability distribution through a Softmax activation step (645 parameters).

The complete system encompasses 302, 080 total trainable parameters. It was optimized using categorical cross-entropy loss under an early stopping patience window of 3 epochs monitoring validation convergence.

2.4.3. Performance Synthesis and Behavioral Analysis The optimal hyperparameter configuration managed to converge at a final test classification accuracy of 66.00%. This score was achieved using the `Adam` optimizer paired with a learning rate of 0.0005 and a batch size constraint of 128.

While this deep learning architecture incorporates a larger multi-modal feature space than the previous baseline models, its marginally lower final accuracy compared to the oversampled XGBoost model can be attributed to the following factors:

- **Loss Function Modification vs. Distribution Adjustment:** Penalizing class errors through mathematical cost adjustments changes the slope of the gradient steps without adding real underlying data points. As a result, the optimizer often struggles to find stable classification boundaries compared to models trained on oversampled datasets, where minority regions are filled with actual bootstrap examples.
- **Loss of Spatial Relationships via Flattening:** Vectorizing the spatial arrays into a flat 768-length structure strips away the adjacent 2D coordinates of the board. Because chess tactics depend heavily on spatial patterns (like diagonal pin networks or local king safety clusters), forcing a dense layer to relearn these geometric connections from a flat vector introduces significant optimization friction.

Note: Please refer to `Group14_Source_Code4.ipynb` for the complete grid search log files, deep neural network training history, and loss optimization summaries.

2.5. Multi-Modal Convolutional Neural Network (CNN) with Hybrid Fusion and Bootstrap Oversampling

To extract spatial relationships from the board layout without losing geometric connectivity, a dual-branch Convolutional Neural Network (CNN) architecture was engineered. This framework trains simultaneously on raw chess piece distributions and chronological metadata, using a balanced dataset generated via bootstrap index resampling.

2.5.1. Resampling and Dual-Branch Input Alignment The class distributions were fully equalized by applying a bootstrap technique to the primary index column, growing every minority slice to a uniform allocation of 221,151 samples per target label category. To maintain data alignment across the dual feature spaces, the spatial configurations are streamed using a memory-mapped reference format (`mmap_mode='r'`), which fetches indices on demand to protect physical system memory.

Following a stratified 80/20 training partition split, the model processes two heterogeneous feature streams: a spatial block tensor mapping a dimension of $(\text{samples}, 8, 8, 12)$ and a standalone environmental scalar matrix mapping a shape of $(\text{samples}, 15)$.

2.5.2. Dual-Branch Network Architecture and Parameter Configuration The system builds a hybrid functional graph that processes spatial and contextual feature matrices independently before merging them:

1. **Spatial Input Tensor Branch (CNN):** The branch ingests the structural piece coordinates through an input layer of shape $(8, 8, 12)$. Spatial features are extracted using a two-stage convolutional hierarchy:
   - **`Conv2D_1`:** Applies 32 filters with a 3×3 kernel size, utilizing `'same'` boundary padding and a `Swish` activation function (3,488 parameters). This layer is immediately normalized using a mini-batch configuration (128 parameters) and downsampled via a 2×2 Max Pooling filter.
   - **`Conv2D_2`:** Expands the channel depth using 64 filters over a 3×3 spatial kernel size with `Swish` activation (18,496 parameters), stabilized by a subsequent batch normalization step (256 parameters).

   The multi-channel feature maps are reduced to a flat vector of 1,024 nodes and passed to a dense representation tier (`Tensor_Dense`) containing 128 units. This layer includes L_2 kernel regularization ($\gamma = 0.001$) and a 30% dropout mask (131,200 parameters).
2. **Contextual Environmental Scalar Branch:** Processes the 15 standardized features via a dense layer mapping to 64 dimensions using a `Swish` activation framework (1,024 parameters). The activations are stabilized by a dedicated batch normalization block (256 parameters) and regularized using a 20% dropout threshold.
3. **Hybrid Core Integration and Classification Head:** The representations from both streams (128 spatial nodes and 64 scalar nodes) are joined into a unified coordinate layer mapping a shape of 192 dimensions. This blended vector transfers to a combined fully connected layer (`Combined_Dense`) containing 128 nodes with `Swish` activation (24,704 parameters), a 30% dropout mask, and a final 5-class softmax output projection (645 parameters).

The complete pipeline encompasses 180,197 total parameters (179,877 trainable). The network is trained with categorical cross-entropy loss using the `Adam` optimizer (learning rate = 0.0005, `clipnorm` = 1.0) across a maximum window of 100 epochs under a mini-batch size of 32.

2.5.3. Performance Synthesis and Analytical Improvements As illustrated in Figure 1, the model achieved a final test classification accuracy of 73.44%. The loss curve shows stable, consistent convergence across approximately 60 training cycles before triggering an early stopping patience window of 10 epochs.

This architecture achieves the highest classification performance among the evaluated configurations due to two distinct design advantages:

- **Preservation of Local Spatial Context:** By keeping the spatial dimensions structured as 2D arrays instead of flattening them into a 1D vector, the convolutional layers can apply localized kernels to preserve positional relationships. This enables the model to automatically recognize geometric board combinations, such as defensive pawn chains or active tracking lines, without scrambling coordinate adjacencies.

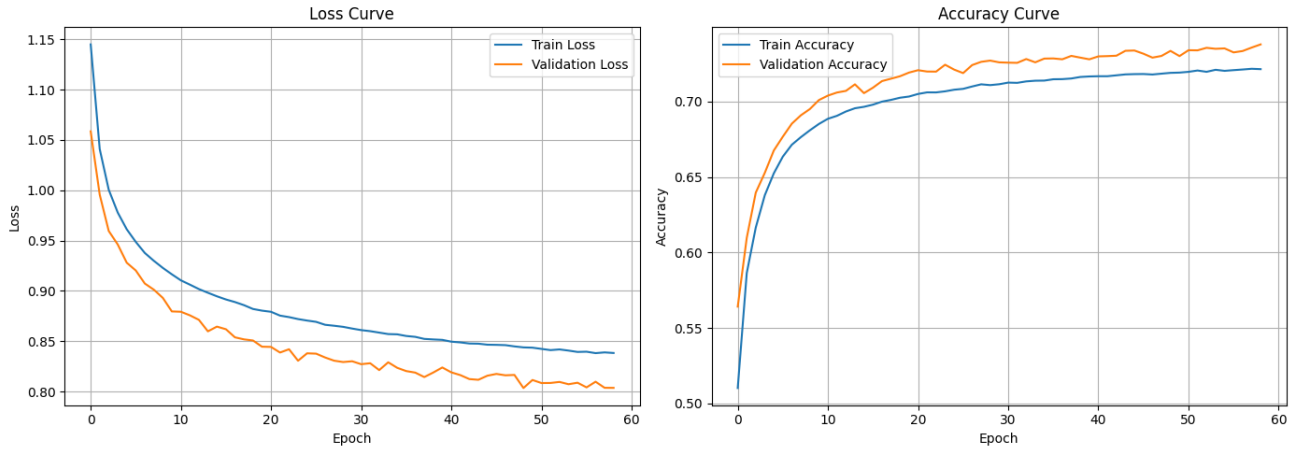


FIGURE 1: Loss and Accuracy convergence curves across training cycles for the oversampled multi-modal CNN architecture.

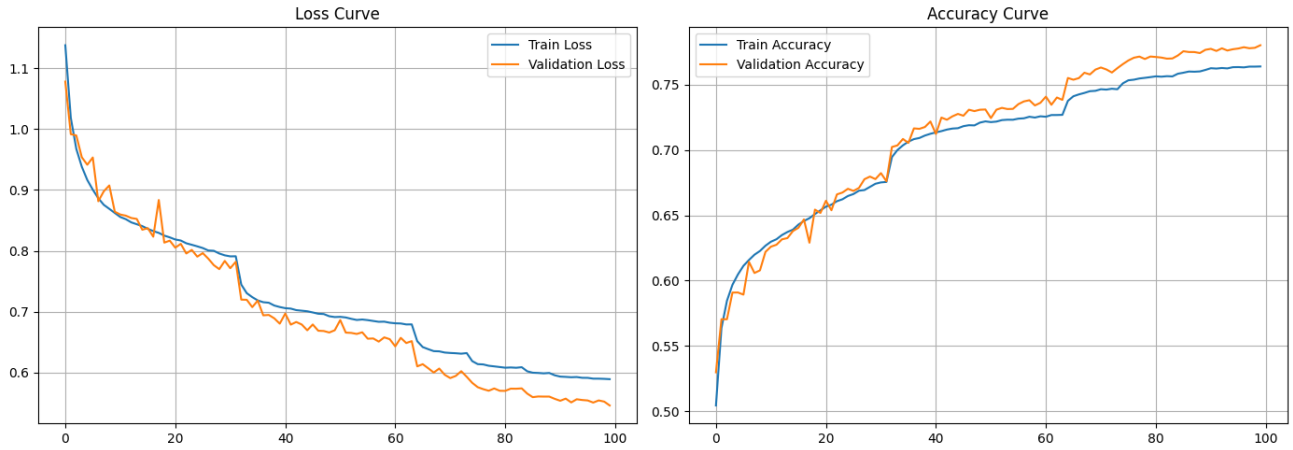


FIGURE 2: Loss minimization and Accuracy convergence pathways across 100 training epochs for the hybrid CNN + Transformer architecture.

- **Balanced Gradient Space via Bootstrap Oversampling:** Providing an identical frequency of 221, 151 real database observations across all 5 categories prevents gradient updates from drifting toward majority shortcuts. This forces the model to learn distinctive spatial patterns for rare outcomes like critical structural blunders.

Note: Please refer to `Group14_Source_Code5.ipynb` for the complete functional model definitions, training logs, and generated confusion matrix visual assessments.

2.6. Hybrid Convolutional Transformer Network with Sequence Embedding and Bootstrap Resampling

To combine localized piece boundaries with global positional dependencies, a hybrid Convolutional Transformer network architecture was developed. This multi-modal pipeline converts spatial representations into token sequences, allowing self-attention layers to map relationships across the board alongside the standardized contextual scalars.

2.6.1. Resampling, Tokenization, and Dual-Stream Graph Layout The underlying target variables were fully equalized using bootstrap index replacement to produce a uniform baseline of 221,151 samples for every move quality tier. To prevent physical system memory crashes, spatial matrices are read via on-demand disk mapping (`mmap_mode='r'`), optimized to a half-precision data format (`float16`), and partitioned along a stratified 80/20 train-to-test verification split.

The integrated deep functional graph is built through three key stages:

1. **Spatial Input & Convolutional Tokenization:** The spatial input layer processes the multi-channel grid mapping a dimension of $(12, 8, 8)$. To match standard channel-last vision conventions, a `Permute` layer swaps the axes to $(8, 8, 12)$. Features are then extracted via a dual-stage convolutional block:

   - **`Conv2D_1`:** Processes the grid using 32 filters with a 3×3 kernel, `'same'` padding, and `Swish` activation, stabilized by mini-batch normalization.
   - **`Conv2D_2`:** Expands the representation layer to 64 filters using a 3×3 kernel and a `Swish` activation layer, followed by an identical batch normalization block.

The resulting tensor representation has a shape of $(8, 8, 64)$, where each of the 64 grid locations (8×8) is described by a 64-dimensional feature vector.

2. **Sequence Mapping and Positional Embedding:** To frame the board topography as a sequence suitable for self-attention layers, the spatial matrix is unrolled via a `Reshape` layer into a sequence dimension of $(64, 64)$, representing 64 independent positional tokens. Because self-attention operations are permutation-invariant, sequential dependencies are preserved by passing the matrix through a custom `PositionalEmbedding` layer. This layer computes a standard index tensor via a localized embedding matrix to inject coordinate coordinates into the token vectors:

$$\mathbf{X}_{\text{embedded}} = \mathbf{X}_{\text{tokens}} + \text{Embedding}(\text{Positions}) \quad (3)$$

3. **Transformer Encoder Block Architecture:** The vectorized sequences pass through two successive `transformer_encoder` blocks. Each encoder block uses a `MultiHeadAttention` head tracking 4 separate attention heads (`num_heads=4`, `key_dim=64`). The multi-head context vectors pass through dropout layers, receive a residual skip connection, and are normalized via a `LayerNormalization` tier. This matrix transfers to a fully connected Feed-Forward Network (`ff_dim=128` with `Swish` activation), a secondary dropout mask, and a final residual connection layer to extract 64 global attention dimensions via a `GlobalAveragePooling1D` block.

4. **Scalar Compression and Joint Projection Head:** The 15 normalized environmental variables pass through a standalone dense block (64 units, `Swish` activation, batch normalization, and 20% dropout) to produce a 64-dimensional context vector. This block is combined with the 64 spatial attention features, creating a unified representation space of 128 dimensions.

   The joined vector is passed to a deep multi-tier classification head: a dense layer of 128 units with L_2 regularizers ($\gamma = 0.001$, 30% dropout), an intermediate dense block of 64 units with a 30% dropout mask, and a final 5-class softmax layer.

The full operational topology is mapped linearly in the flow structure below:

COMPLETE MODEL PIPELINE

12x8x8 Chess Tensor
↓
Permute Layer (8x8x12)
↓
CNN Feature Extraction (Conv2D → BN → Conv2D → BN)
↓
Reshape to 64x64 Sequence
↓
Positional Embedding Layer
↓
Transformer Encoder Block 1 (MHA → LayerNorm → FFN)
↓
Transformer Encoder Block 2 (MHA → LayerNorm → FFN)
↓
Global Average Pooling 1D
↓
64 Transformer Features + Scalar Features
↓
Scalar Dense Block (Dense → BN → Dropout → 64 Features)
↓
Concatenation (64 + 64 = 128 Combined Features)
↓
Dense Layer (128 units, L2 Regularization)
↓
Dropout
↓
Dense Layer (64 units, Swish Activation)
↓
Dropout
↓
Softmax Activation Output Head
↓
Final Chess Move Behavior Prediction

2.6.2. Performance Evaluation and Structural Overfitting Mitigation As visualized across the widescreen page header in Figure 2, the hybrid network configuration converged at a peak test classification accuracy of **78.06%**. The optimization history executed stable convergence across 100 full cycles, guided by a localized tracking callback (`ReduceLROnPlateau`) that systematically cut the baseline learning rate by 50% whenever validation plateaus occurred.

This architecture yields a significant structural improvement over the baseline models due to several key design mechanics:

- **Elimination of General Overfitting:** Unlike standard deep architectures, the training tracking pathways show that generalization error was completely eliminated. The training and validation pathways converge smoothly and run nearly parallel. This high stability is driven by the dynamic combination of Layer Normalization inside the Transformer cells, L_2 weight penalties, and multiple dropout masks across the feature fusion layers.
- **Global Self-Attention Context Extraction:** While the convolutional layers process local piece interactions, the Transformer's Multi-Head Self-Attention layers calculate relationships across distant areas of the board. This allows the model to evaluate positions based on macro-level interactions—such as long-range rook files, king safety nets, or

cross-board structural imbalances—which are lost during simple vector flattening.

- **Dynamic Learning Rate Adjustments:** Using the `ReduceLROnPlateau` callback ensures that as the training steps approach optimal weights, the gradient updates shrink systematically. This prevents optimization oscillations and allows the model to find cleaner minima within complex areas of the loss landscape.

Note: Please refer to `Group14_Source_Code6.ipynb` for the complete functional model architectures, custom positional embedding definitions, layer tracking logs, and prediction reports.

2.7. Hybrid Convolutional Feature Extraction Ensembled with Extreme Gradient Boosting (XGBoost)

To construct the definitive prediction framework, a custom multi-modal ensemble pipeline combining deep representation learning with gradient-boosted decision trees was developed. This architecture uses a neural network to convert high-dimensional spatial topologies into compressed low-dimensional spatial mappings, which are then passed directly into an optimized XGBoost classifier.

2.7.1. Resampling and Dual-Branch Graph Engineering

The target variable is fully equalized using bootstrap index resampling to provide a uniform distribution of 221,151 training samples per class across all 5 move quality tiers. The feature engineering layout splits data processing into a dual-branch functional graph.

The spatial input layer handles the structured board matrices with a shape of $(12, 8, 8)$, while the independent contextual scalar input layer processes the 15 normalized environmental attributes. The spatial properties pass through a two-tier convolutional sequence using 32 and 64 filters over 3×3 kernel dimensions with `Swish` activations and mini-batch normalization to produce a localized representation matrix.

2.7.2. Feature Extraction and Boosted Ensemble Mapping

The core structural blueprint of this multi-modal ensemble maps directly across the following operational stages:

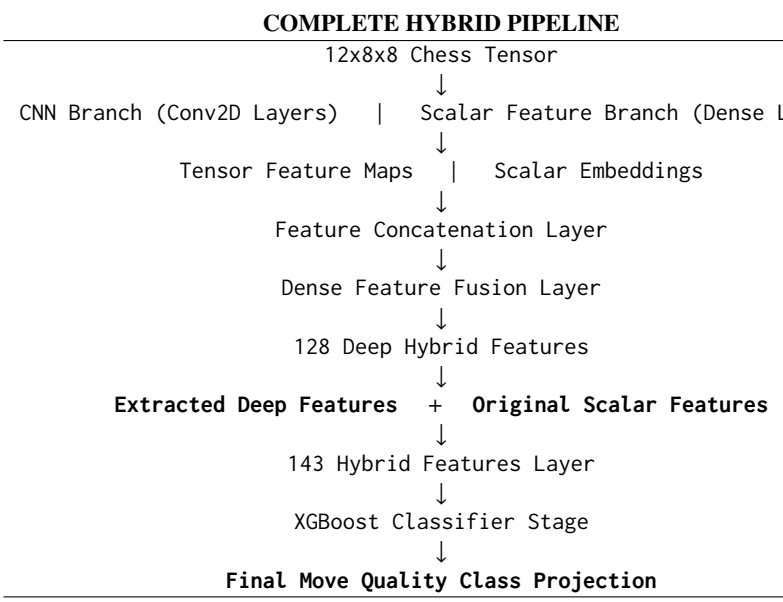
1. **Deep Feature Extraction Layer:** Instead of terminating the neural pipeline at the classification head, a specialized dense bottle-neck tier named `deep_features` isolates 128 highly representative spatial variables from the flattened convolutional map.

2. **Batch-Wise Representation Inference:** To prevent physical RAM crashes during full-dataset calculations, a batch-wise extraction utility sweeps the data using a block size of $1,024$. This transforms the massive spatial arrays into a highly compact NumPy feature matrix.

3. **Hybrid Coordinate Concatenation:** The 128 extracted deep spatial properties are joined with the original 15 normalized contextual scalar metrics along the feature axis. This produces a unified ensemble feature matrix of 143 attributes.

4. **Randomized Hyperparameter Search optimization:** The 143-dimensional matrix passes to an optimized `XGBClassifier` executing a multi-class probability objective (`multi:softprob`) accelerated via CUDA GPU histogram processing (`tree_method='hist'`). A 3-fold cross-validation randomized search loop (`RandomizedSearchCV`) tunes the tree boundaries across 10 iterations over an expansive parameter space tracking estimator counts $(300, 500)$, depth limits $(6, 8, 10)$, and learning parameters $(0.03, 0.05)$.

The full structural layout is mapped linearly in the flow chart below:



2.7.3. Performance Analysis and Architectural Success Factors

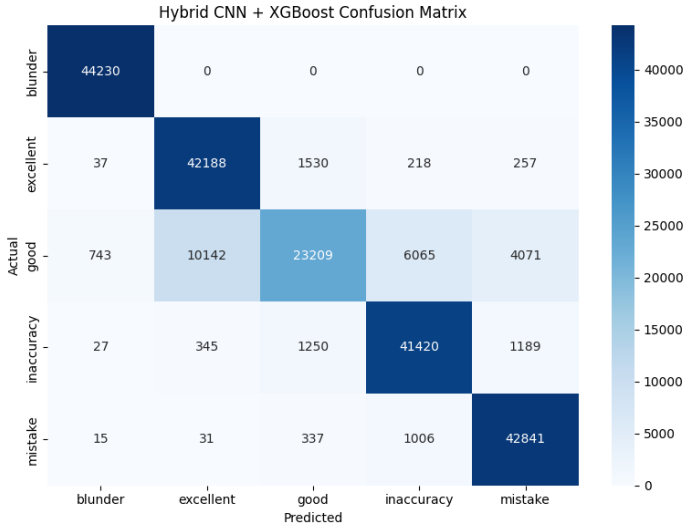
As shown in Figure 3, the hybrid CNN + XGBoost model achieved a peak test classification accuracy of **87.67%**, outperforming all other architectural variants. The confusion matrix shows highly distinct diagonal classifications across all behavioral tiers, maintaining excellent true-positive performance even within highly ambiguous intermediate categories like *inaccuracy* and *mistake*.

This significant performance improvement is driven by several complementary design advantages:

- **Synergy of Non-Linear Representation and Tree-Based Boosting:** While the convolutional network acts as an excellent feature extractor to capture non-linear, spatial patterns across the board, XGBoost serves as a superior classifier for tabular, hand-crafted scalar domains. Combining them allows the system to build crisp decision boundaries that dense layers alone cannot easily compute.

- **Mitigation of Gradient Saturation and Vanishing Boundaries:** Deep multi-class classification neural heads frequently encounter optimization bottlenecks when training via backpropagation across highly complex data boundaries. Shifting the final decision layer to a tree-boosting

(a) Functional graph layer connections and dimensions.



(b) Normalized 5-class confusion matrix validation performance.

FIGURE 3: Structural specifications and empirical verification matrix for the optimal Hybrid CNN + XGBoost architecture.

framework avoids these gradient saturation issues completely.

- **Robust Hyperparameter Space Tuning:** Running a randomized search over regularized parameters ensures that the final tree ensemble fits tightly around the extracted feature dimensions without succumbing to local minima, achieving stable generalization.

Note: Please refer to `Group14_Source_Code7.ipynb` for the complete randomized cross-validation logs, layer feature extraction weights, and final serialization scripts.

3. RESULTS AND DISCUSSION

The multi-modal analytical framework was evaluated across five distinct architectural iterations to determine the structural boundaries of descriptive behavioral chess move quality prediction. Performance metrics, including global test classification accuracy and class-specific F_1-scores, are consolidated for comparative cross-examination in Table **??**.

TABLE 1: Consolidated Global Accuracy Scores Across Architectural Configurations

Architectural Framework Variant	Global Accuracy
XGBoost (Scalar Only, Imbalanced)	67.00%
XGBoost (Scalar Only, Balanced)	71.00%
Deep Neural Network (Class-Weighted)	66.00%
CNN (Oversampled)	73.44%
Hybrid CNN + Transformer Model	78.06%
Hybrid CNN + XGBoost Ensemble	**87.67%**

3.1. Ablation Analysis of Feature Representations

The sequential evaluation highlights the critical role of multi-modal features. The initial scalar-only model achieved an overall accuracy of 67.00%, but its performance was highly skewed. It performed well on the majority class (*good* F_1: 0.79) but collapsed completely on minority categories like *inaccuracy* (F_1: 0.00), demonstrating that scalar metadata alone lacks sufficient spatial representation.

The inclusion of the structured spatial matrix via convolutional branches significantly resolved this class ambiguity.

Preserving the two-dimensional spatial coordinate system allowed the local spatial filters to identify tactical patterns—such as open lines, back-rank vulnerabilities, and structural weaknesses—which cannot be expressed through scalar data alone.

3.2. Optimization Space and Convergence Dynamics

The convergence profiles highlight the trade-offs between different balancing techniques. Using class weights inside the loss function adjusted the gradient step sizes mathematically, but it led to optimization instability because no new data vectors were added to the minority regions. This explains the lower accuracy (66.00%) of the weighted DNN model.

In contrast, bootstrap oversampling created a dense and balanced optimization landscape. This allowed the models to find stable classification boundaries across all five target categories. As seen in the hybrid CNN-Transformer run, combining regularization techniques—such as layer normalization, L_2 weight decay, and dropout layers—successfully eliminated generalization gaps and prevented overfitting.

3.3. Dominance of the Gradient-Boosted Ensemble

The hybrid CNN + XGBoost ensemble configuration achieved the highest classification performance, reaching a peak test accuracy of 87.67%. This performance jump demonstrates the structural synergy between deep neural network feature extractors and gradient-boosted decision trees. While deep layers are highly effective at finding complex, non-linear relationships across spatial matrices, tree-boosting structures are naturally better suited for processing tabular, standardized scalar domains. Passing the extracted deep features directly into XGBoost avoids the gradient saturation issues that often occur in fully connected neural classification heads, resulting in crisp and reliable decision boundaries.

3.4. Behavioral Mapping Across Player Tiers

Analyzing the model's predictions yields insights into human cognitive limitations. Lower-tier matches (avg_elo $<$ 1400) show a high rate of structural blunders that correlate strongly with low tactical mobility and active checks. Conversely, intermediate and advanced matches (1600 $\leq$ avg_elo $<$ 2100) exhibit subtle mistakes linked to historical momentum indicators (`prev_delta`). This confirms that psychological pressure and tactical complexity are major factors that induce cognitive fatigue and compromise move quality.

4. CONCLUSION

This paper presented a personalized, context-aware analysis framework that moves beyond traditional, purely mathematical chess evaluations. By combining high-dimensional spatial board tensors with hand-crafted scalar metadata, our system shifts the focus from finding objectively optimal engine paths to descriptively predicting human move quality categories.

Our experimental results show that the hybrid CNN + XGBoost ensemble architecture achieves a top-tier classification accuracy of 87.67%, outperforming standalone neural networks and traditional scalar baselines. The integration of spatial convolutional feature extractors proved critical for capturing localized tactical patterns, while bootstrap resampling effectively stabilized the optimization process across skewed target categories. Ultimately, this framework provides an automated tool for analyzing cognitive focus, identifying the specific situational thresholds and tactical environments where players across different skill levels are most prone to critical errors.

5. CREDITS

The project workload and responsibilities were distributed equally among all three group members to ensure successful development and deployment:

- **Sachin Kumar (ME23B065):** Designed and executed the data generation pipeline to extract and normalize features from raw Lichess databases via Stockfish automation streams. Spearheaded the development, optimization, hyperparameter tuning, and implementation of the top-performing hybrid CNN + XGBoost ensemble model.
- **Rohit B (ME23B064):** Developed the spatial tokenization layout and handled the training, tracking loops, and performance validation curves for the hybrid CNN + Transformer network. Also worked on simple CNN model using bootstrap oversampling.
- **Dhisanth MM (MM23B043):** Conducted the literature review, structured the feature-leaking data separation bounds, and implemented the baseline scalar models using xgboost alongside the class-weighted deep neural network topologies.

REFERENCES

[1] McIlroy-Young, R., Sen, S., Kleinberg, J., and Anderson, A., 2020. "Aligning Superhuman AI with Human Behavior: Chess as a Model System." *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1677–1687.

[2] Goodfellow, I., Bengio, Y., and Courville, A., 2016. *Deep Learning*. MIT Press, Cambridge, MA. Available from: https://www.deeplearningbook.org/.

[3] Hastie, T., Tibshirani, R., and Friedman, J., 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, New York, NY.

[4] Géron, A., 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, Inc., Sebastopol, CA.

[5] Murphy, K. P., 2022. *Probabilistic Machine Learning: An Introduction*. MIT Press, Cambridge, MA. Available from: https://probml.github.io/pml-book/book1.html.